Отчет по лабораторной работе

Тема: Разработка интерактивного конвертера цветовых моделей (RGB, CMYK, HLS)

1 Цель работы

Изучение принципов представления цвета в цифровых системах, разработка программного приложения для многостороннего преобразования между тремя цветовыми моделями: **RGB** (Red, Green, Blue), **CMYK** (Cyan, Magenta, Yellow, Key/Black) и **HLS** (Hue, Lightness, Saturation), а также реализация интерактивного пользовательского интерфейса с мгновенной синхронизацией элементов управления.

2 Ход работы и архитектура приложения

Разработанное веб-приложение построено по классической трехфайловой архитектуре (разделение ответственности):

1. `index.html` — разметка страницы, содержащая блоки («карточки») для каждой из трех цветовых моделей, состоящие из пар элементов управления `<input type="range»` и `<input type="number»`, а также блок выбора цвета с помощью стандартной палитры (`<input type="color»`).
2. `style.css` — таблицы стилей, использующие технологию **Flexbox** для адаптивного выравнивания блоков в ряд и построения элементов управления.
3. `script.js` — скрипт, реализующий математические алгоритмы конвертации цветов между всеми тремя моделями через базовое цветовое пространство RGB, управление состоянием приложения и синхронизацию элементов интерфейса.

3 Математическое обеспечение и алгоритмы конвертации

В качестве центрального базового пространства для всех промежуточных вычислений выбрана модель **RGB**, где значения каналов заданы в диапазонах $R, G, B \in [0, 255]$.

3.1 Преобразование RGB $\leftrightarrow$ CMYK

Аддитивная модель RGB преобразуется в субтрактивную модель CMYK ($C, M, Y, K \in [0, 100]\%$) следующим образом:

1. Нормализация каналов RGB к диапазону $[0, 1]$:

$$r_c = \frac{R}{255}, \quad g_c = \frac{G}{255}, \quad b_c = \frac{B}{255}$$

2. Вычисление черного ключа (K):

$$K = 1 - \max(r_c, g_c, b_c)$$

Примечание: Если $K = 1$ (абсолютный черный цвет), то составляющие рассчитываются как $C = 0, M = 0, Y = 0, K = 100$. В противном случае:

3. Расчет основных красок:

$$\begin{aligned} C &= \left(\frac{1 - r_c - K}{1 - K}\right) \times 100, \\ M &= \left(\frac{1 - g_c - K}{1 - K}\right) \times 100, \\ Y &= \left(\frac{1 - b_c - K}{1 - K}\right) \times 100, \\ K &= K \times 100 \end{aligned}$$

Обратный перевод из CMYK в RGB выполняется по формулам:

$$R = 255\left(1 - \frac{C}{100}\right)\left(1 - \frac{K}{100}\right), \quad G = 255\left(1 - \frac{M}{100}\right)\left(1 - \frac{K}{100}\right), \quad B = 255\left(1 - \frac{Y}{100}\right)\left(1 - \frac{K}{100}\right)$$

3.2 Преобразование RGB $\leftrightarrow$ HLS

Модель HLS описывает цвет через тон ($H \in [0, 360°]$), яркость ($L \in [0, 100]\%$) и насыщенность ($S \in [0, 100]\%$).

1. Нормализация и поиск экстремумов:

$$r' = \frac{R}{255}, \quad g' = \frac{G}{255}, \quad b' = \frac{B}{255}, \quad M_{\max} = \max(r', g', b'), \quad M_{\min} = \min(r', g', b')$$

2. Расчет яркости (L) и насыщенности (S):

$$L = \frac{M_{\max} + M_{\min}}{2}$$

$$S = \begin{cases} 0, & \text{если } M_{\max} = M_{\min} \\ \frac{M_{\max} - M_{\min}}{2 - M_{\max} - M_{\min}}, & \text{если } L > 0.5 \\ \frac{M_{\max} - M_{\min}}{M_{\max} + M_{\min}}, & \text{если } L \le 0.5 \end{cases}$$

3. Расчет цветового тона (H):

$$H = \begin{cases} 0, & \text{если } M_{\max} = M_{\min} \\ 60° \times \left(\frac{g' - b'}{M_{\max} - M_{\min}} \pmod 6\right), & \text{если } M_{\max} = r' \\ 60° \times \left(\frac{b' - r'}{M_{\max} - M_{\min}} + 2\right), & \text{если } M_{\max} = g' \\ 60° \times \left(\frac{r' - g'}{M_{\max} - M_{\min}} + 4\right), & \text{если } M_{\max} = b' \end{cases}$$

Обратный переход из HLS в RGB реализуется через вспомогательные промежуточные коэффициенты p и q, зависящие от яркости и насыщенности, с последующим сопоставлением для каждого канала по формулам порогового пересчета зон цветового круга.

4 Листинг разработанного программного кода

4.1 index.html (фрагмент структуры)

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>                                </title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="preview-container">
        <div class="palette-box">
            <label for="colorPalette">
               </label><br>
            <input type="color" id="colorPalette" style="width: 150px
               ; height: 40px; cursor: pointer; margin-top: 5px;">
        </div>
        <div>                            :</div>
        <div id="colorBox"></div>
    </div>
    <div class="container">
        <!--RGB-->
        <div class="model-card" id="rgbCard">
            <h3>RGB</h3>
            <div class="control-group">
                <label>Red (0-255)</label>
                <div class="control-row">
                    <input type="range" min="0" max="255" value="255"
                        data-model="rgb" data-prop="r">
                    <input type="number" min="0" max="255" value="255
                       " data-model="rgb" data-prop="r">
                </div>
            </div>
            <div class="control-group">
                <label>Green (0-255)</label>
                <div class="control-row">
                    <input type="range" min="0" max="255" value="255"
                        data-model="rgb" data-prop="g">
                    <input type="number" min="0" max="255" value="255
                       " data-model="rgb" data-prop="g">
                </div>
            </div>
            <div class="control-group">
                <label>Blue (0-255)</label>
                <div class="control-row">
                    <input type="range" min="0" max="255" value="255"
                        data-model="rgb" data-prop="b">
                    <input type="number" min="0" max="255" value="255
                       " data-model="rgb" data-prop="b">
                </div>
            </div>
```

```
        </div>
        <!--CMYK-->
        <div class="model-card" id="cmykCard">
            <h3>CMYK</h3>
            <div class="control-group">
                <label>Cyan (0-100%)</label>
                <div class="control-row">
                    <input type="range" min="0" max="100" value="0"
                       data-model="cmyk" data-prop="c">
                    <input type="number" min="0" max="100" value="0"
                       data-model="cmyk" data-prop="c">
                </div>
            </div>
            <div class="control-group">
                <label>Magenta (0-100%)</label>
                <div class="control-row">
                    <input type="range" min="0" max="100" value="0"
                       data-model="cmyk" data-prop="m">
                    <input type="number" min="0" max="100" value="0"
                       data-model="cmyk" data-prop="m">
                </div>
            </div>
            <div class="control-group">
                <label>Yellow (0-100%)</label>
                <div class="control-row">
                    <input type="range" min="0" max="100" value="0"
                       data-model="cmyk" data-prop="y">
                    <input type="number" min="0" max="100" value="0"
                       data-model="cmyk" data-prop="y">
                </div>
            </div>
            <div class="control-group">
                <label>Key/Black (0-100%)</label>
                <div class="control-row">
                    <input type="range" min="0" max="100" value="0"
                       data-model="cmyk" data-prop="k">
                    <input type="number" min="0" max="100" value="0"
                       data-model="cmyk" data-prop="k">
                </div>
            </div>
        </div>
        <!--HLS-->
        <div class="model-card" id="hlsCard">
            <h3>HLS</h3>
            <div class="control-group">
                <label>Hue (0-360  )</label>
                <div class="control-row">
                    <input type="range" min="0" max="360" value="0"
                       data-model="hls" data-prop="h">
                    <input type="number" min="0" max="360" value="0"
                       data-model="hls" data-prop="h">
                </div>
            </div>
```

```
            <div class="control-group">
                <label>Lightness (0-100%)</label>
                <div class="control-row">
                    <input type="range" min="0" max="100" value="100"
                        data-model="hls" data-prop="l">
                    <input type="number" min="0" max="100" value="100
                       " data-model="hls" data-prop="l">
                </div>
            </div>
            <div class="control-group">
                <label>Saturation (0-100%)</label>
                <div class="control-row">
                    <input type="range" min="0" max="100" value="0"
                       data-model="hls" data-prop="s">
                    <input type="number" min="0" max="100" value="0"
                       data-model="hls" data-prop="s">
                </div>
            </div>
        </div>
    </div> <!--container-->
<script src="script.js"></script>
</body>
</html>
```

Листинг 1: HTML разметка интерфейса для трех моделей

4.2 style.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f4f4f9;
    color: #333;
    margin: 0;
    padding: 20px;
    display: flex;
    flex-direction: column;
    align-items: center;
}

h1 {
    margin-bottom: 20px;
    color: #2c3e50;
}

.container {
    display: flex;
    gap: 20px;
    flex-wrap: wrap;
    justify-content: center;
    max-width: 1000px;
}

```

```
.model-card {
    background: #ffffff;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    width: 280px;
}

.model-card h3 {
    margin-top: 0;
    border-bottom: 2px solid #eee;
    padding-bottom: 8px;
    color: #34495e;
    text-align: center;
}

.control-group {
    margin-bottom: 14px;
}

.control-group label {
    display: block;
    font-size: 13px;
    font-weight: bold;
    margin-bottom: 4px;
    color: #555;
}

.control-row {
    display: flex;
    gap: 10px;
    align-items: center;
}

.control-row input[type="range"] {
    flex-grow: 1;
    cursor: pointer;
}

.control-row input[type="number"] {
    width: 60px;
    padding: 4px;
    text-align: center;
    border: 1px solid #ccc;
    border-radius: 4px;
}

.preview-container {
    margin-bottom: 25px;
    background: #ffffff;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
```

```
    width: 880px;
    text-align: center;
}

.palette-box {
    margin-bottom: 10px;
}

#colorPalette {
    border: none;
    width: 150px;
    height: 80px;
    border-radius: 8px;
    cursor: pointer;
    background: transparent;
}

#colorPalette::-webkit-color-swatch-wrapper {
    padding: 0;
}

#colorPalette::-webkit-color-swatch {
    border: 2px solid #ccc;
    border-radius: 6px;
}

#colorBox {
    width: 100%;
    height: 80px;
    border-radius: 6px;
    border: 1px solid #ccc;
    margin-top: 10px;
}
```

Листинг 2: Таблицы стилей CSS

4.3 script.js

```
// Current machine color
let currentColor = { r: 60, g: 145, b: 235};

function rgbToCmyk(r, g, b) {
    let rc = r / 255, gc = g / 255, bc = b / 255;
    let k = 1 - Math.max(rc, gc, bc);

    if (k === 1) return {c: 0, m: 0, y: 0, k: 100};

    let c = (1 - rc - k)/(1 - k);
    let m = (1 - gc - k)/(1 - k);
    let y = (1 - bc - k)/(1 - k);

    return {
        c: Math.round(c * 100),
```

```
        m: Math.round(m * 100),
        y: Math.round(y * 100),
        k: Math.round(k * 100)
    };
}

function cmykToRgb(c, m, y, k) {
    c = c / 100, m = m / 100, y = y / 100, k = k / 100;
    let r = 255 * (1 - c) * (1 - k);
    let g = 255 * (1 - m) * (1 - k);
    let b = 255 * (1 - y) * (1 - k);
    return {
        r: Math.round(r),
        g: Math.round(g),
        b: Math.round(b)
    };
}

function hlsToRgb(h, l, s) {
    h /= 360; l /= 100; s /= 100;
    let r, g, b;

    if (s === 0) {
        r = g = b = l; // achromatic
    } else {
        const hue2rgb = (h, m1, m2) => {
            if (h < 0) h += 1;
            if (h > 1) h -= 1;
            if (h < 1/6) return m1 + (m2 - m1) * 6 * h;
            if (h < 1/2) return m2;
            if (h < 2/3) return m1 + (m2 - m1) * (2/3 - h) * 6;
            return m1;
        };
        let m2 = l < 0.5 ? l * (1 + s) : l + s - l*s;
        let m1 = 2*l - m2;
        r = hue2rgb(h + 1/3, m1, m2);
        g = hue2rgb(h,       m1, m2);
        b = hue2rgb(h - 1/3, m1, m2);
    }
    return {
        r: Math.round(r * 255),
        g: Math.round(g * 255),
        b: Math.round(b * 255)
    };
}

function rgbToHls(r, g, b) {
    r /= 255; g /= 255; b /= 255;
    let max = Math.max(r, g, b);
    let min = Math.min(r, g, b);
    let h, s, l = (max + min) / 2;

    if (max === min) {
```

```
        h = s = 0; // achromatic;
    } else {
        let d = max - min;
        s = l>0.5 ? d/(2-max-min) : d / (max + min);
        switch (max) {
            case r: h = (g - b) / d + (g < b ? 6 : 0); break;
            case g: h = (b - r) / d + 2; break;
            case b: h = (r - g) / d + 4; break;
        }
        h /= 6;
    }
    return {
        h: Math.round(h * 360),
        l: Math.round(l * 100),
        s: Math.round(s * 100)
    };
}

function updateUI(sourceModel) {
    let cmyk = rgbToCmyk(currentColor.r, currentColor.g, currentColor
       .b);
    let hls  = rgbToHls(currentColor.r, currentColor.g, currentColor.
       b);

    // Same vars as data-model in index.html
    let values = {
        rgb: currentColor,
        cmyk: cmyk,
        hls: hls
    };

    // Select all <input> elements in doc which have
    // data-model attribute and apply assignment of values.
    document.querySelectorAll("input[data-model]").forEach(input => {
            let model = input.dataset.model;
            let prop = input.dataset.prop;
            if (model !== sourceModel) {
                input.value = values[model][prop];
            }
    });

    // Update colorBox background style with new color.
    document.getElementById('colorBox').style.background =
    `rgb(${currentColor.r}, ${currentColor.g}, ${currentColor.b})`;

    // Update colorPalette internal hex value.
    let hex = "#" + ((1 << 24) + (currentColor.r << 16) + (
       currentColor.g << 8) + (currentColor.b)).toString(16).slice(1)
       ;
        document.getElementById('colorPalette').value = hex;
}

// Add listeners to all range- and number-inputs.
```

```
document.querySelectorAll('input[data-model]').forEach(input => {
    input.addEventListener('input', (e) => {
        let model = e.target.dataset.model;
        let prop = e.target.dataset.prop;
        let val = parseInt(e.target.value) || 0;

        // Sync range- and corresponding number- inputs.
        let card = e.target.closest('.model-card');
        card.querySelectorAll(`input[data-prop="${prop}"]`).forEach(
            el => {
                el.value = Math.max(el.min, Math.min(el.max, val));
            }
        )

        // Select current model values, which is being selected.
        let inputs = card.querySelectorAll('input[type="number"]');
        let data = {};
        inputs.forEach(inp => {
            data[inp.dataset.prop] = Math.max(inp.min, Math.min(inp.
               max,
                parseInt(inp.value) || 0
            ));
        })

        if (model === 'rgb') {
            currentColor = { r: data.r, g: data.g, b: data.b };
        } else if (model === 'cmyk') {
            currentColor = cmykToRgb(data.c, data.m, data.y, data.k);
        } else if (model === 'hls') {
            currentColor = hlsToRgb(data.h, data.l, data.s);
        }

        updateUI(model);
    });
});

// Add listener to the palette.
document.getElementById('colorPalette').addEventListener('input', (e)
    => {
    let hex = e.target.value;
    let r = parseInt(hex.substr(1, 2), 16);
    let g = parseInt(hex.substr(3, 2), 16);
    let b = parseInt(hex.substr(5, 2), 16);
    currentColor = {r, g, b};

    updateUI('palette'); // Will update all models.
});

updateUI('init'); // Will update and sync all models.
```

Листинг 3: Логика приложения для RGB, CMYK и HLS

5 Заключение

В ходе выполнения лабораторной работы были успешно освоены принципы перевода цветовых пространств между аддитивной (RGB), субтрактивной (CMYK) и перцептивной (HLS) моделями. Реализовано модульное веб-приложение с использованием современных стандартов разметки, стилизации (Flexbox) и интерактивного управления состоянием через событийно-ориентированный JavaScript. Приложение полностью функционально и обеспечивает мгновенную двустороннюю синхронизацию данных между всеми тремя цветовыми пространствами. Реализовывать подобные приложения с использованием JavaScript, html, css оказалось достаточно быстро и, вероятно, легче, чем, например, полноценное приложение (с использованием Qt, Python, etc.). Преобразования RGB - CMYK являются более интуитивными и однозначными, в то время как преобразование RGB - HLS содержит множество ветвлений и отображение одного трёхмерного множества точек в другое трехмерное. За основу была взята модель RGB (т.к. она аппаратно зависимая, а все остальные модели пересчитываются через неё).